Práctica 3. Microservicios Avanzados y BBDD NoSQL

Versión 3.6 - 23/10/2026

Introducción

En esta práctica se aplicarán los conocimientos adquiridos en prácticas anteriores para implementar una arquitectura basada en microservicios más completa y profesional. Para ello se utilizarán diferentes tecnologías y sistemas gestores de bases de datos NoSQL.

Objetivos docentes

Tras la realización de esta práctica, el estudiante debe ser capaz de:

- Afianzar conocimientos sobre creación de aplicaciones distribuidas basadas en API REST y microservicios.
- Afianzar conocimiento sobre el manejo de contenedores para la creación de microservicios.
- Adquirir conocimientos sobre el diseño de arquitecturas distribuidas avanzadas.
- Adquirir conocimientos sobre el uso de bases de datos NoSQL, como Redis y MongoDB, integradas dentro de aplicaciones basadas en microservicios.

Tecnologías a utilizar

Para esta práctica, se utilizará:

- Nginx [1] como servidor proxy inverso, funcionando como API Gateway.
- Quart [2] como framework ASGI (Asynchronous Server Gateway Interface) e Hypercorn [3] como servidor ASGI. Hypercorn será en este caso el servidor predeterminado, es decir, la opción recomendada para desplegar aplicaciones Quart en producción. Su uso con Quart proporciona robustez y además tiene algunas ventajas, como la recarga automática integrada (opción `--reload` en Dockerfile) cuando detecta cambios en el código, reiniciando el servidor al instante. Además, permite la depuración interactiva, mostrando errores detallados directamente en la terminal o el navegador al fallar.
- Docker [4] como sistema de virtualización basado en contenedores para distribuir los servicios.
- MongoDB [5] como gestor de base de datos NoSQL. Se utilizará la API asíncrona oficial de PyMongo mediante `pymongo.AsyncMongoClient`. Todas las operaciones de entrada/salida con MongoDB deberán realizarse de forma asíncrona.

- Redis [6] como gestor de base de datos NoSQL. Se usará la librería `redis-py` que incorpora soporte para trabajar con conexiones asíncronas (`redis.asyncio`).

Al igual que en prácticas anteriores, las tecnologías de soporte se desplegarán en contenedores. Además, se implementará un único fichero `docker-compose.yml` para levantar y detener todos los contenedores y servicios necesarios.

Tareas a realizar

Se desea crear un proyecto que implemente un sistema distribuido y desacoplado basado en una arquitectura de microservicios de tipo API REST. El proyecto contendrá dos tipos de microservicios diferentes: uno para la **autenticación de usuarios** basada en tokens (*auth-service*), y otro para la **gestión de libros y préstamos de una biblioteca** (*library-service*), cada uno con sus endpoints correspondientes que se describirán a continuación.

1. Microservicio de usuarios con persistencia NoSQL

Este microservicio se apoyará en Nginx, que se utilizará como punto de entrada, evitando que el cliente tenga que conocer las direcciones de cada contenedor y centralizando así la seguridad. Los microservicios no deben ser accesibles directamente desde el exterior; el único punto de entrada debe ser Nginx. Es decir, se usará, como Gateway inteligente, de forma que el API Gateway recibe la petición con el token, pregunta al microservicio de **autenticación de usuarios** si es válido y, si lo es, redirige la petición al microservicio para la **gestión de libros y préstamos de una biblioteca**.

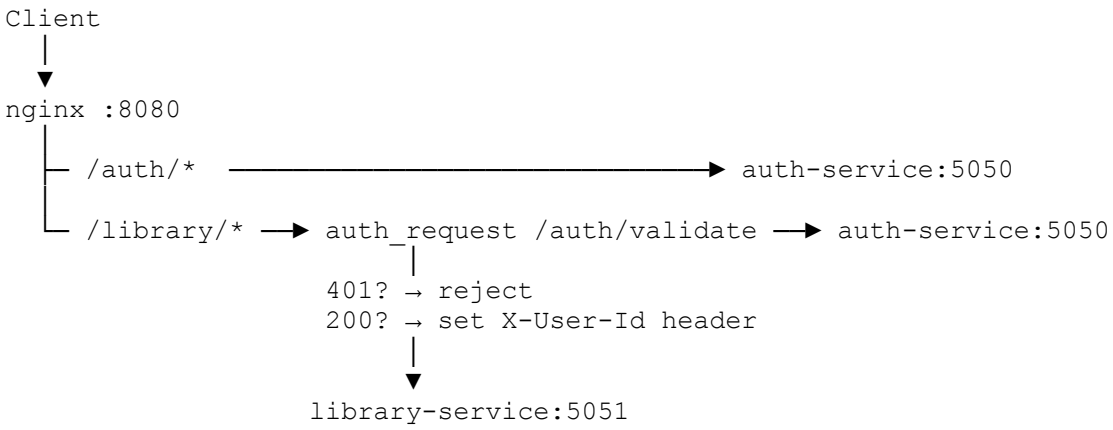
Para la gestión de datos y persistencia se usará el sistema gestor de bases de datos NoSQL llamado **Redis**, con persistencia (AOF) para gestionar los usuarios y sus tokens. De esta forma, Redis asume un doble rol: el almacenamiento permanente de las credenciales de los usuarios (gracias a AOF), y el almacenamiento temporal de los tokens de sesión con TTL (tiempo de expiración).

El microservicio contendrá cuatro endpoints principales, de los cuales sólo los dos primeros son públicos, ya que `…/auth/validate`, que acepta el encabezado "`Authorization: Bearer <token>`", busca "`session:{token}`" en Redis, y devuelve `username/user_id` o el código de error 401 indicando que el usuario no tiene permisos para realizar la operación. Este endpoint es necesario para que pueda ser llamado por `auth_request` de Nginx. También se implementará la función de logout para usuarios que hayan hecho login previamente. Este endpoint debe lee el token `Bearer` y eliminar `session:{token}` en Redis, devolviendo 200 si la operación es exitosa, o 401 si falta la cabecera, el token es desconocido o ha caducado, o si se intenta cerrar sesión nuevamente. El token se invalida de inmediato, en lugar de esperar a que transcurra el TTL.

Endpoint	Método	Descripción
`…/auth/register`	POST	Registra usuarios (username, password y email).
`…/auth/login`	POST	Inicial la sesión de un usuario registrado (username, password), devolviendo su token correspondiente.
`…/auth/validate`	GET	Llamado internamente por nginx auth_request. Devuelve 200 + encabezado X-User-Id si la operación se realiza correctamente, o 401 si falla.
`…/auth/logout`	POST	Cierra la sesión del usuario e invalida el token actual.

Respecto al modelo de datos, los perfiles de usuario permanentes se almacenarán en estructuras de tipo HASH bajo la clave `user:{username}`, incluyendo campos como `password_hash` y `email`. Al ser datos persistentes, no tendrán tiempo de expiración (TTL) y se guardarán en disco mediante la directiva `--appendonly yes` en el archivo YAML. Por otro lado, las sesiones temporales utilizarán claves de tipo STRING o HASH con el formato `session:{token}` para vincularlas al usuario. Estas claves incluirán un TTL de 5 minutos, garantizando que Redis las elimine automáticamente tras su vencimiento.

Nginx se utiliza frecuentemente como proxy inverso, balanceador de carga y caché. Es ampliamente utilizado para gestionar el tráfico de sitios web grandes debido a su bajo consumo de memoria y alta velocidad. Un solo proceso puede manejar miles de conexiones simultáneas sin sobrecargar la memoria del servidor. De esta forma, Nginx llama a `/auth/validate` antes de redirigir la solicitud al microservicio de la biblioteca. Dicho servicio confía en un encabezado inyectado (`X-User-Id`) establecido por Nginx. Esto proporciona un desacoplamiento limpio, pero requiere sub-solicitudes de Nginx. De hecho, Nginx funciona como puerta de enlace API: para las rutas protegidas del microservicio de biblioteca, utiliza la directiva `auth_request` para validar el token con el servicio de autenticación antes de redirigir la solicitud al microservicio de la biblioteca, de forma que captura las rutas de la forma:



2. Microservicio para la gestión de libros y préstamos de una biblioteca

Para el microservicio de **gestión de libros y préstamos de una biblioteca**, se usará el sistema gestor de bases de datos NoSQL MongoDB. El modelo de datos estará basado en una colección de libros llamada "books", cuyos documentos representan libros (book) que tendrán la siguiente estructura:

```
[{
  isbn: String,
  title: String,
  author: String,
  year: Integer,
  publisher: String,
  available_stock: Integer,
  "loan_history": [
    {
      user_id: String,
```

```
      loan_date: datetime,
      return_date: datetime
    },
    …
  ]
}, …]
```

Cada libro tiene un sub-documento embebido que representa el historial de préstamos que ha tenido a lo largo de su historia (`loan_history`). Las operaciones a implementar (endpoints) por el microservicio de **gestión de libros y préstamos de una biblioteca** son las siguientes:

Endpoint	**Método**	**Descripción**
…/library/books	POST	Crea un nuevo libro.
…/library/books/<isbn>	DELETE	Borra un libro existente. Se debe controlar el caso de borrar un libro que tiene préstamos activos (no permitiendo que se pueda borrar hasta su devolución).
…/library/books/<isbn>	GET	Obtiene información de un libro, así como la lista de préstamos.
…/library/loans/active	GET	Lista los libros que están prestados al usuario (autenticado).
…/library/loans/history	GET	Lista el histórico de préstamos del usuario (autenticado).
…/library/books/out-of-stock	GET	Lista los libros que tengan stock 0.
…/library/books/<isbn>/borrow	POST	Toma en préstamo un libro por parte de un usuario (autenticado).
…/library/books/<isbn>/return	POST	Devuelve un libro que estaba prestado a un usuario (autenticado).

Los endpoints anteriores estarán protegidos, y sólo podrán acceder a ellos los usuarios que hayan iniciado sesión previamente, por lo que la información para ciertas operaciones se tomará de dicho usuario con sesión (token) activa. El histórico para un libro, cuando se crea el libro, estará vacío (array vacío). Cuando se realiza un préstamo, se actualiza el préstamo creando una nueva entrada (sub-documento). En ese caso, la fecha de retorno valdrá inicialmente NULL hasta que se devuelva el libro, momento en el que se actualizará con la fecha del día de devolución. Además, hay que actualizar el campo `available_stock` cuando se presta y se devuelve un libro.

Es posible que dos peticiones concurrentes puedan observar stock disponible y terminar dejando `available_stock` por debajo de cero. Por ello, es mejor que el préstamo y la devolución se implementen mediante una operación atómica de MongoDB, no aceptando secuencias de lectura-comprobación-escritura que permitan *condiciones de carrera*. Para ello, se debe utilizar `find_one_and_update()` con filtro atómico.

El microservicio de biblioteca no accede a Redis ni valida tokens. Nginx realiza una sub-petición a `auth-service`; si la validación es correcta, elimina cualquier valor recibido del cliente para los encabezados de identidad e inyecta un encabezado interno `X-User-Id`. El microservicio de biblioteca confía únicamente en ese encabezado, dado que no es accesible directamente desde el exterior.

3. Poblado y testeo para la comprobación de la práctica

Se proporciona un fichero de poblado (`populate.json`) para poblar la BD de MongoDB que se debe usar obligatoriamente, de forma que se pueda inicializar automáticamente la BD a partir del fichero `__init__.py`.

Además, se proporciona una batería de test de tipo pytest [8] que realiza peticiones a los distintos servicios y realiza las siguientes comprobaciones:

- Se registra un usuario correctamente.
- Se autentica un usuario correctamente.
- Se crea un libro correctamente.
- Se realiza un préstamo sobre un libro correctamente.
- Se lista correctamente todos los libros en préstamo del usuario autenticado.
- Se devuelve el libro prestado por el usuario autenticado correctamente.
- Comprueba que un usuario ya no tiene el libro en préstamo tras la devolución.
- Comprueba que no se puede realizar un acceso a la biblioteca sin token.
- Comprueba que no se puede acceder al historial de otro usuario.
- Comprueba que no se puede realizar un acceso a la biblioteca sin token válido.
- Comprueba que no se pueda borrar un libro con préstamo activo.
- Comprueba que el campo de stock no puede tomar valores negativos (*condición de carrera*)
- Comprueba condiciones de login/logout

La práctica implementada debe satisfacer los test anteriores. Por tanto, se debe modificar el código del estudiante, no los test.

Es importante no cambiar los nombres de elementos, objetos y especificaciones indicados en este enunciado.

4. Estructura del proyecto

Se seguirá obligatoriamente la siguiente distribución de ficheros y carpetas a la hora de crear, organizar y entregar el proyecto:

```
P3/
│
├── api-gateway/
│   └── nginx.conf
│
├── auth-service/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── redis_client.py
│   │   └── routes.py
│   ├── Dockerfile
│   └── requirements.txt
│
├── library-service/
│   ├── app/
│   │   ├── __init__.py
```

```
│   │   ├── populate.json
│   │   ├── mongodb_client.py
│   │   └── routes.py
│   ├── Dockerfile
│   └── requirements.txt
├── docker-compose.yml
├── pytest.ini
├── p3_test.py
├── p3_test_requirements.txt
└── README.txt
```

La descripción general de estos ficheros es la siguiente:

- **`api-gateway/nginx.conf`**: Fichero de configuración del servicio de Nginx, que recibe las peticiones en el puerto 8080 y decide si la petición es para auth-service o para library-service.
- **`auth-service/app/__init__.py`**: Inicializa la aplicación Quart.
- **`auth-service/app/routes.py`**: Contiene los endpoints para registrar usuarios de forma permanente en Redis y gestionar el login/tokens temporales, así como la validación para Nginx.
- **`auth-service/app/redis_client.py`**: Archivo para configurar el cliente de Redis asíncrono y poder importarlo en las rutas.
- **`library-service/app/__init__.py`**: Inicializa la aplicación Quart de la biblioteca (library-service) y abre la conexión asíncrona hacia el contenedor de MongoDB. También, aquí se tendría que ejecutar la rutina de poblado de la base de datos en MongoDB, que tome los datos del fichero `populate.json` y los inserte en la BD indicada, a través de alguna lógica asíncrona (`@app.before_serving`, o similar).
- **`library-service/app/populate.json`**: Contiene documentos, en formato JSON, con los que poblar la base de datos anteriormente mencionada.
- **`library-service/app/mongodb_client.py`**: Archivo para configurar el cliente de MongoDB asíncrono y poder importarlo en las rutas.
- **`library-service/app/routes.py`**: Contiene los endpoints para gestionar libros y préstamos del microservicio de la biblioteca.
- **`docker-compose.yml`**: Fichero para configurar y levantar todos los servicios simultáneamente (nginx, mongodb, redis, auth-service, library-service).
- **`Dockerfile` y `requirements.txt`**: Representan los ficheros para crear una imagen de Docker de las aplicaciones (`Dockerfile`) y los ficheros de dependencias (`requirements.txt`), respectivamente, de cada microservicio.
- **`p3_test.py`**: Representa un fichero de test en pytest que comprueba si se realiza la funcionalidad esperada. Se ha creado también un fichero **pytest.ini** a efectos de configuración, que contiene: "`[pytest] addopts = -p no:asyncio`". El fichero `p3_test_requirements.txt` que se encuentra en la raíz del proyecto (P3) contiene las dependencias para ejecutar el test de `pytest`.

Se recomienda el uso de un entorno virtual para instalar las dependencias de los test y ejecutarlos. Por otro lado, el proyecto completo debe poder ser arrancado y detenido a través del comando `docker compose`, usando el fichero YAML ubicado en la raíz del proyecto. La configuración del

proyecto debe ser lo suficientemente flexible como para cambiar fácilmente los puertos en caso de que alguno esté ocupado. Por último, para realizar las comprobaciones oportunas sobre los endpoints, se recomienda el uso del comando `curl` [7] desde la terminal de comandos.

Se recomienda también el uso de volúmenes externos (locales) para las bases de datos de Redis y MongoDB, de forma que se pueda asegurar la persistencia cuando no estén activos los contenedores.

5. Memoria

Se entregará una memoria explicativa y bien documentada que incluya las decisiones de diseño tomadas para cada tarea.

Planificación

Se recomienda la siguiente planificación para desarrollar la práctica:

Semana 1	Instalación y configuración de los servicios
Semana 2	Microservicio de autenticación
Semana 3	Microservicio de biblioteca
Semana 4	Testing
Resto de días	Memoria, finalización y entrega

Entrega

La fecha concreta y normas de entrega de las prácticas se encuentran en Moodle. Como mínimo se deben entregar los siguientes archivos:

- Se entregará un único fichero ZIP (llamado `P3.zip`) que contenga el proyecto completo, con todos los ficheros necesarios para su ejecución. Esto incluye los ficheros fuente en python, scripts, dependencias, fichero YAML, etc., todo ello organizado correctamente en carpetas y sub-carpetas, según lo indicado más arriba, junto con un fichero README.txt en la raíz con información e instrucciones básicas de despliegue y ejecución local. No se deben entregar entornos virtuales, cachés, bytecode ni metadatos del sistema operativo.
- Memoria de la práctica con una descripción y capturas de la resolución de cada apartado.

Rúbrica de corrección

Se seguirá la siguiente rúbrica para la calificación de la práctica:

Concepto	Calificación Máxima
Calidad del proyecto y de la configuración de los servicios	2
Implementación del microservicio de autenticación	3
Implementación del microservicio de la biblioteca	4
Memoria	1
TOTAL	**10**

Se aplicarán la penalización de 1 punto por día natural (o fracción) de retraso en la entrega. De la misma forma, se sancionará, según la normativa de la UAM, la copia o el uso indiscriminado de la IA.

Bibliografía

[1] Nginx Docs: https://nginx.org/en/docs/

[2] Quart Docs: https://quart.palletsprojects.com/en/latest/

[3] Hypercorn Docs: https://hypercorn.readthedocs.io/en/latest/

[4] Docker Docs: https://docs.docker.com

[5] MongoDB Docs: https://www.mongodb.com/es/docs/

[6] Redis Docs: https://redis.io/docs/latest/

[7] Curl Docs: https://curl.se/docs/manpage.html

[8] Pytest Docs: https://docs.pytest.org/en/stable/